Textbook # chapter 3.1 | LECTURE - 15 | Monday 26/10/26

Slides: Lecture 19-20_ Compiler level...

☐ Loop Unrolling (ILP: chapter 3.2): Reduce no. of stalls by unrolling loops & then scheduling

floating point arithmetic

```
for (i = 999; i ≥ 0; i--) {
    x[i] = x[i] + s;
}
```

Base address of array 'x' i.e: 999

```
1 fld   f0, 0(x1)     // f0 = x[999] load
2 fadd.d f4, f0, f2   // f2 is 's'
3 fsd   f4, 0(x1)     // store in x[i]
4 addi  x1, x1, -8    // decrement double word
5 bne   x1, x2, loop  // keep looping
```

* 1, 2, 3 lines are dependent. 4, 5 are independent.

Pearl Notes

CamScanner

Date

- we'll observe the instruction in risc-V overlapping pair wise like : 1 & 2, 2 & 3, 3 & 4 ...

	Instr. producing result	Instru. using result	Latency in Clock cycle
a	FP ALU OP	Another FP ALU OP	3
b	FP ALU OP	Store double	2
c	Load Double	FP ALU OP	1
d	Load Double	Store double.	0

- 1st & 2nd Instruction: (1 is load double) & (2 is FP ALU OP (add) code.)
∴ we'll have 'c' latency from above table i.e: [1] No operation
- 2nd & 3rd : (2 is FP ALU OP) & (3 is store double)
∴ we'll have 'b' latency so Latency = [2]

Total cycles : 5 were used simply for 5 instruction.
2+1 [3] new stalls because of latency table
∴ Total Cycles = [8].

⇒ Reducing these cycles: we'll do it by unrolling the loop. In it we'll use jumps after hardcoding the same code n no. of times. 'n' is decided by compiler.
- Now we'll write the depending instruction (1, 2, 3) three times as n=3 & only once we'll write (4,5) independent instruc.
- New registers will be used for every hardcoded cycle.

CamScanner

NOTE: Doing this unrolling increases lines of code which puts pressure on cache & increases memory.

Date

A		B	
Loop: fld	f0, 0(x1)	fld	
fadd.d	f4, 0, f2	fld	Instru.
fsd	f4, 0(x1)	fld	1
fld	f6, -8(x1)	add.d	This is
fadd.d	f8, f6, f2	add.d	of
fsd	f8, -8(x1)	add.d	Instruction
fld	f10, -16(x1)	fsd	2
fadd.d	f12, f10, f2	fsd	
fsd	f12, -16(x1)	fsd	
addi	x1, x1, x-24	addi	
bne	x1, x2, Loop	bne	

Explanation of A:

- we've written the hardcoded loop 3 times unroll kr liay & all the underlined with pink registers are diffirent
- The decremet is -8 → -16 and then -24 when addi as addi is done once.
- This wasn't removing stalls as we still have NOP from tables like. FP ALU after load double etc.

Explanation of B:

- we combined all the similar instructions & ran them together so there is a gap btw "add.d" instruction of Instr#1 and "fld" instruction of Instr#1.
- All fld first, All add.d second, All fsd third.
- This removes all NOP

CamScanner